

Similarity-Based Duplicate Record Detection System

DATA SCIENCE (24AD3107) Project Review-1



Team Members

Allipuram Akhil – 2420030540
P. Ruthvik Reddy – 2420030546
Surya – 2420030440



Under the Guidance of

Ms. Bonagani Prathusha

Assistant Professor,

Department of Computer Science and Engineering,

KLH CSE Bowrampet Campus

Outline of Project Work

- Focus

- Introduction

- Literature Survey

- Challenges

- Research Objectives

- Conclusion & Future Work

- References

- Publications Details

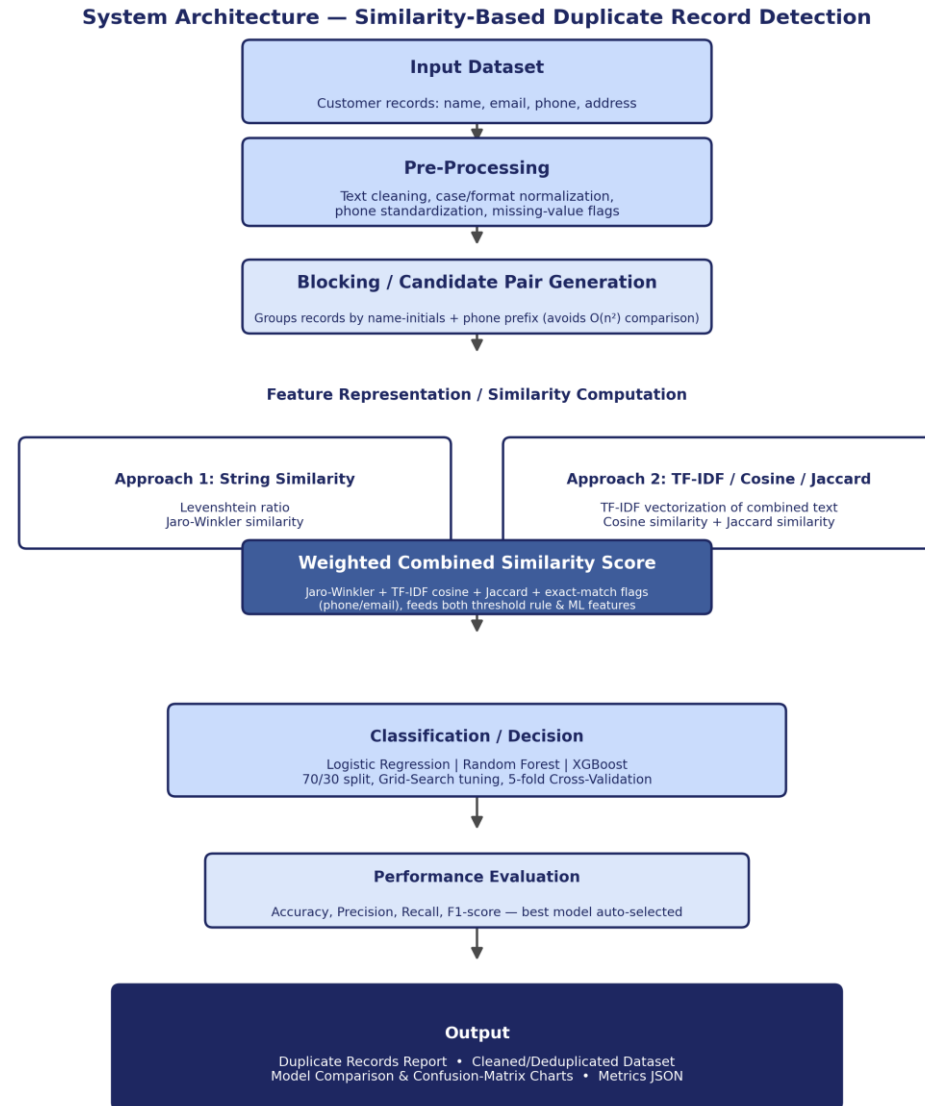
Problem Statement

- Organizations collect customer and record data from multiple sources (web forms, system migrations, merges), which introduces exact and near-duplicate records that differ due to typos, formatting, or missing fields.
- Manual identification of duplicate records is time-consuming, error-prone, and does not scale to large datasets.
- Naively comparing every record against every other record is computationally expensive — an $O(n^2)$ operation — and becomes impractical as data volume grows.
- There is a need for an automated system that detects both exact and near-duplicate records using similarity measures and machine learning, and produces a clean, deduplicated dataset.

Objectives

- Design a pipeline that pre-processes and standardizes raw record data — names, emails, phone numbers, and addresses.
- Implement an efficient blocking strategy that reduces the candidate-pair search space instead of comparing every record pair.
- Compute complementary similarity measures — string-based (Levenshtein ratio, Jaro-Winkler) and vector-based (TF-IDF, Cosine, Jaccard) — and combine them into a weighted similarity score.
- Train and compare supervised ML classifiers (Logistic Regression, Random Forest, XGBoost) for duplicate / non-duplicate classification, with grid-search hyperparameter tuning.
- Evaluate models using accuracy, precision, recall, and F1-score, and generate a cleaned, deduplicated output dataset.
- Provide an interactive web interface for running detection, inspecting flagged pairs, and visualizing results.

Proposed System Architecture



Literature Survey

Authors	Title of the Paper	Methodology	Inferences	Limitation/Future work
Elmagarmid, Ipeirotis & Verykios [1]	Duplicate Record Detection: A Survey, IEEE TKDE, 2007	Comparative survey of field-matching, string-similarity, rule-based, and blocking techniques for duplicate detection	Establishes the blocking + similarity + classification taxonomy that most modern pipelines, including this one, still follow	Survey pre-dates deep-learning approaches; does not benchmark ML classifiers on large modern datasets
Ebraheem et al. [2]	DeepER – Deep Entity Resolution (DeepER), arXiv, 2017	Distributed tuple representations via word embeddings, similarity scoring, and a classifier for entity resolution	Deep representation learning reduces manual feature engineering compared to hand-crafted similarity features	High computational cost and needs large labelled training data; less interpretable than similarity-based scoring
Ravikanth, Korra & Mamidisetti [3]	An Efficient Learning-Based Approach for Automatic Record Deduplication with Benchmark Datasets, Scientific Reports, 2024	LSTM classifier combined with attribute-level similarity metrics, handling missing/null values across records	Attribute-level weighting with a hybrid similarity + deep-learning classifier improves precision/recall over rule-based baselines	Higher computational overhead; accuracy can degrade on highly noisy or partially missing records

Problem Gap Identification

- Traditional duplicate detection relies on exact matching or simple rule-based string comparison, which misses near-duplicates caused by typos, reordering, or missing fields.
- Pure similarity-threshold approaches lack the discriminative power of a trained classifier and require manual threshold tuning for every new dataset.
- Many existing systems compare record pairs exhaustively, which does not scale to large, real-world datasets.
- Few systems combine blocking, multiple similarity approaches, and ML classification together with an interactive, explainable interface for verifying flagged pairs — this is the gap this project addresses.

Innovation, Creativity, and Novelty

- Combines two complementary similarity approaches — string similarity (Levenshtein / Jaro-Winkler) and vector-based similarity (TF-IDF / Cosine / Jaccard) — into one weighted combined score, rather than relying on a single measure.
- Uses a blocking step (name-initial + phone-prefix key) so the pipeline scales to large datasets, mirroring standard practice in real-world entity resolution.
- Learns the decision boundary with ML classifiers, grid-search tuning, and 5-fold cross-validation, instead of a manually fixed similarity threshold.
- Applies union-find based cleaning to keep the most complete record from each duplicate cluster while producing a clean, deduplicated dataset.
- Ships as an interactive Flask web application with side-by-side pair comparison, live progress tracking, and downloadable reports — not just a command-line script.

Feasibility and Project Plan

The project is executed across six phases over the semester, from requirement study through final testing and documentation.

- Phase 1 (Weeks 1–2): Requirement study, literature survey, and dataset design.
- Phase 2 (Weeks 3–4): Pre-processing and blocking implementation.
- Phase 3 (Weeks 5–6): Feature engineering — string similarity, TF-IDF/Cosine/Jaccard, and the combined score.
- Phase 4 (Weeks 7–8): ML model training, hyperparameter tuning, and evaluation.
- Phase 5 (Weeks 9–10): Web interface development and pipeline integration.
- Phase 6 (Weeks 11–12): Testing, documentation, and final report / viva preparation.